TPO

Integrantes

- Juan Pablo Farinola
- Manuel Otero

Propuesta

Propuesta C - Torneo FutbolStast

Requisitos y supuestos

Requisitos funcionales

1. RF01: El sistema debe permitir administrar torneos, clubes, jugadores, entrenadores, árbitros y estadios.
2. RF02: El sistema debe permitir programar partidos, verificando la disponibilidad del estadio.
3. RF03: El sistema debe permitir registrar eventos durante un partido, como goles, cambios y tarjetas.
4. RF04: El sistema debe permitir generar reportes estadísticos y calcular proyecciones de rendimiento.

Requisitos no funcionales

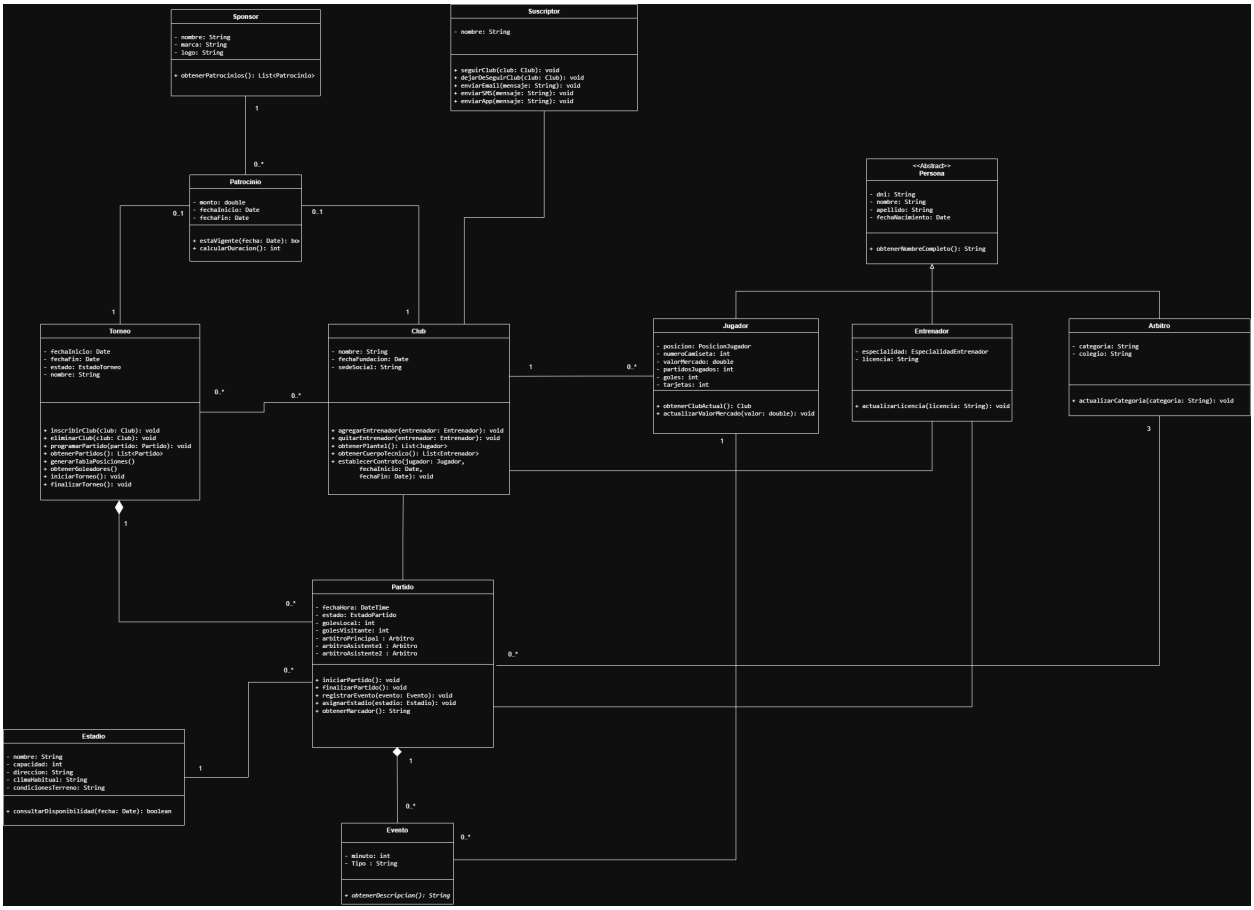
1. RNF01: El sistema debe contar con una interfaz intuitiva y fácil de utilizar.
2. RNF02: El sistema debe garantizar la integridad y consistencia de los datos registrados.

Supuestos

1. S01: Los usuarios cuentan con acceso al sistema y permisos acordes a su rol.

2. S02: Los clubes, jugadores, árbitros y estadios deben estar registrados previamente para poder programar un partido.
3. S03: Se considera que la información ingresada por los usuarios es correcta y está actualizada.
4. S04: El sistema dispone de conexión a internet para enviar las notificaciones a los suscriptores.

Diagrama de Clase Inicial



Principios GRASP

1. Polimorfismo

Se definió la interfaz `CanalNotificaciones`, implementada por las clases `Email`, `App` y `Sms`, cada una con su propia implementación de `enviar()`.

El sistema puede utilizar distintos canales mediante una interfaz común y agregar nuevos canales sin modificar la lógica que los utiliza.

2. Controlador

Se aplicó el principio **Controlador** mediante la incorporación de la clase GestorTorneo, cuya responsabilidad es recibir y coordinar las solicitudes relacionadas con la administración de torneos.

Esta clase actúa como intermediaria entre las solicitudes del sistema y la clase Torneo, delegando operaciones como inscribir clubes, programar partidos, iniciar y finalizar torneos.

De esta manera, se logra una mejor distribución de responsabilidades, evitando que la clase Torneo deba gestionar directamente las solicitudes externas. Además, se mantiene separada la coordinación de las operaciones de la lógica de negocio, lo que facilita el mantenimiento y la modificación del sistema.

3. Bajo acoplamiento

Se aplicó el principio de Bajo Acoplamiento mediante la incorporación de la clase Contrato, que representa el vínculo contractual entre un club y un jugador. Esta clase permite centralizar los datos correspondientes a dicha relación, evitando que las clases Club y Jugador deban almacenar directamente toda la información contractual. De esta manera, se favorece la separación de responsabilidades y se facilita el mantenimiento del sistema.

Principios SOLID

1) Abierto/Cerrado OCP

Se aplica mediante la clase abstracta `Evento` y sus subclases `Gol`, `Cambio`, `Falta`, `Tarjeta` y `Remate`. Esta estructura permite incorporar nuevos tipos de eventos deportivos mediante herencia y polimorfismo, sin modificar las clases existentes. Cada subclase implementa el método `obtenerDescripcion()` según su comportamiento particular.

2) Principios de Inversión de Dependencias (DIP) y Segregación de Interfaces (ISP)

Se aplican mediante la interfaz `CanalNotificaciones`, implementada por las clases `Email`, `App` y `Sms`. Por un lado, DIP se cumple porque la clase `Suscriptor` depende de una abstracción en lugar de depender directamente de las clases concretas, permitiendo incorporar nuevos canales sin modificar su lógica. Por otro lado, ISP se respeta porque la interfaz define únicamente el método `enviar()`, necesario para todos los canales, evitando que las clases deban implementar métodos que no utilizan. De esta manera, se consigue un diseño más flexible, modular y fácil de mantener.

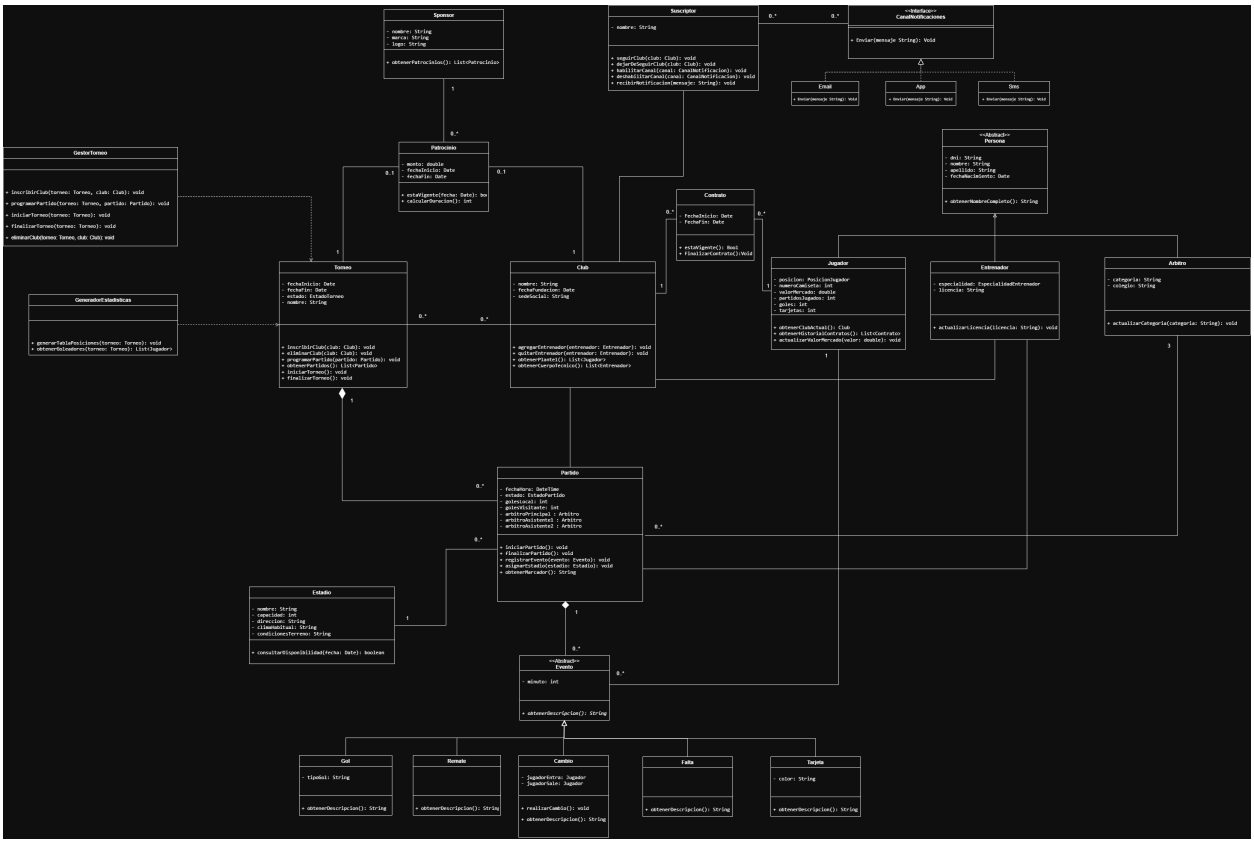
3) Principio de Responsabilidad Única (SRP)

Se aplica mediante la separación de las clases `GestorTorneo` y `GeneradorEstadisticas`. Cada una tiene una responsabilidad específica: `GestorTorneo` se encarga de administrar los torneos, mientras que `GeneradorEstadisticas` se ocupa de generar estadísticas deportivas. De esta manera, los cambios en una responsabilidad no afectan a la otra.

4) Principio de Inversión de Dependencias (DIP)

Se aplica mediante la clase `Suscriptor` y la interfaz `CanalNotificaciones`. La clase `Suscriptor` depende de una abstracción en lugar de depender directamente de las clases concretas `Email`, `Sms` y `App`. Esto permite agregar nuevos canales de notificación sin modificar la clase `Suscriptor`, logrando un sistema más flexible y fácil de mantener.

Diagrama de Clases Final



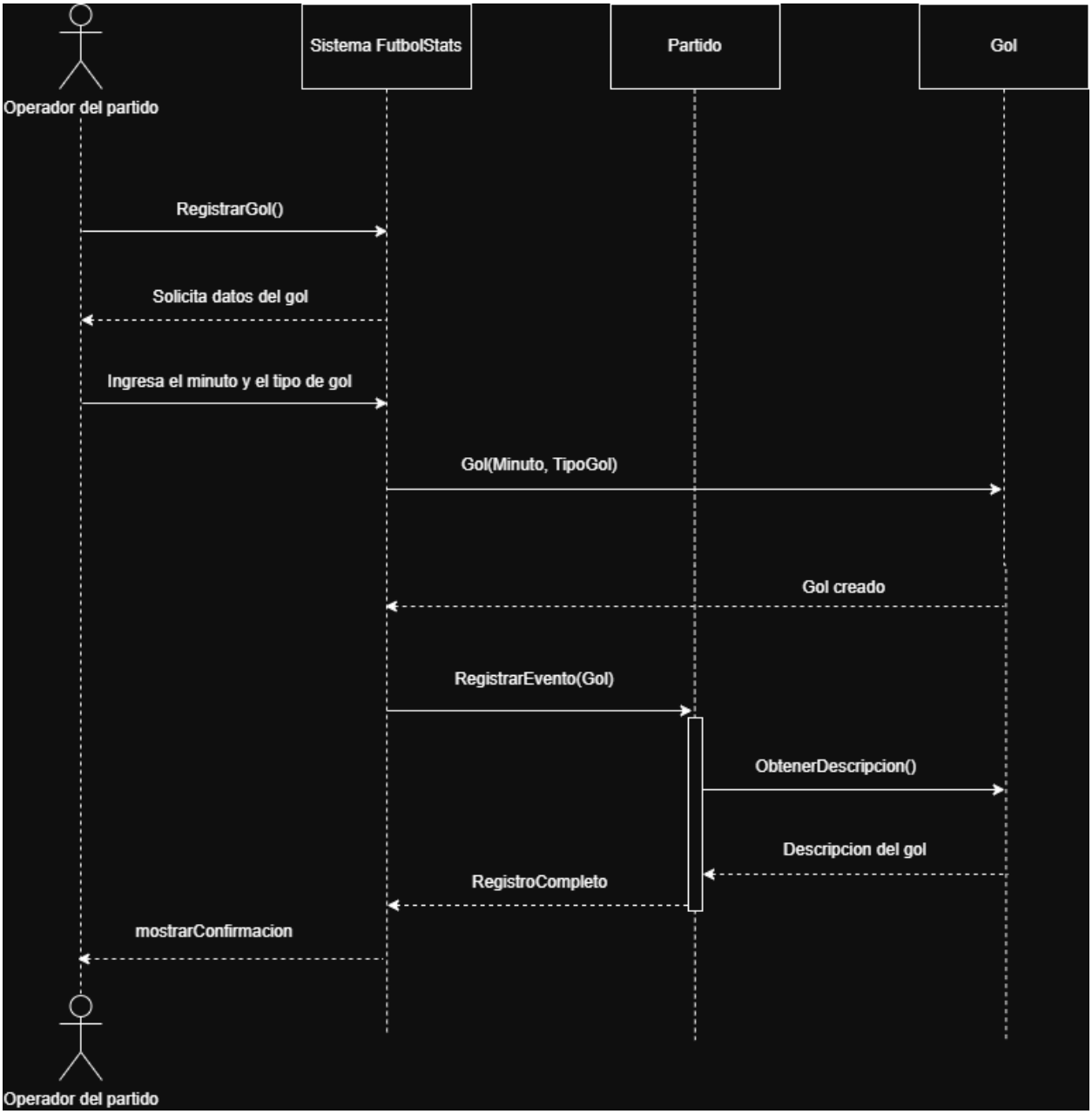
Caso de uso

> Administrar Entidades deportivas : Se refiere a administrar los jugadores, entrenadores, clubes, arbitros , etc



Diagrama de secuencia

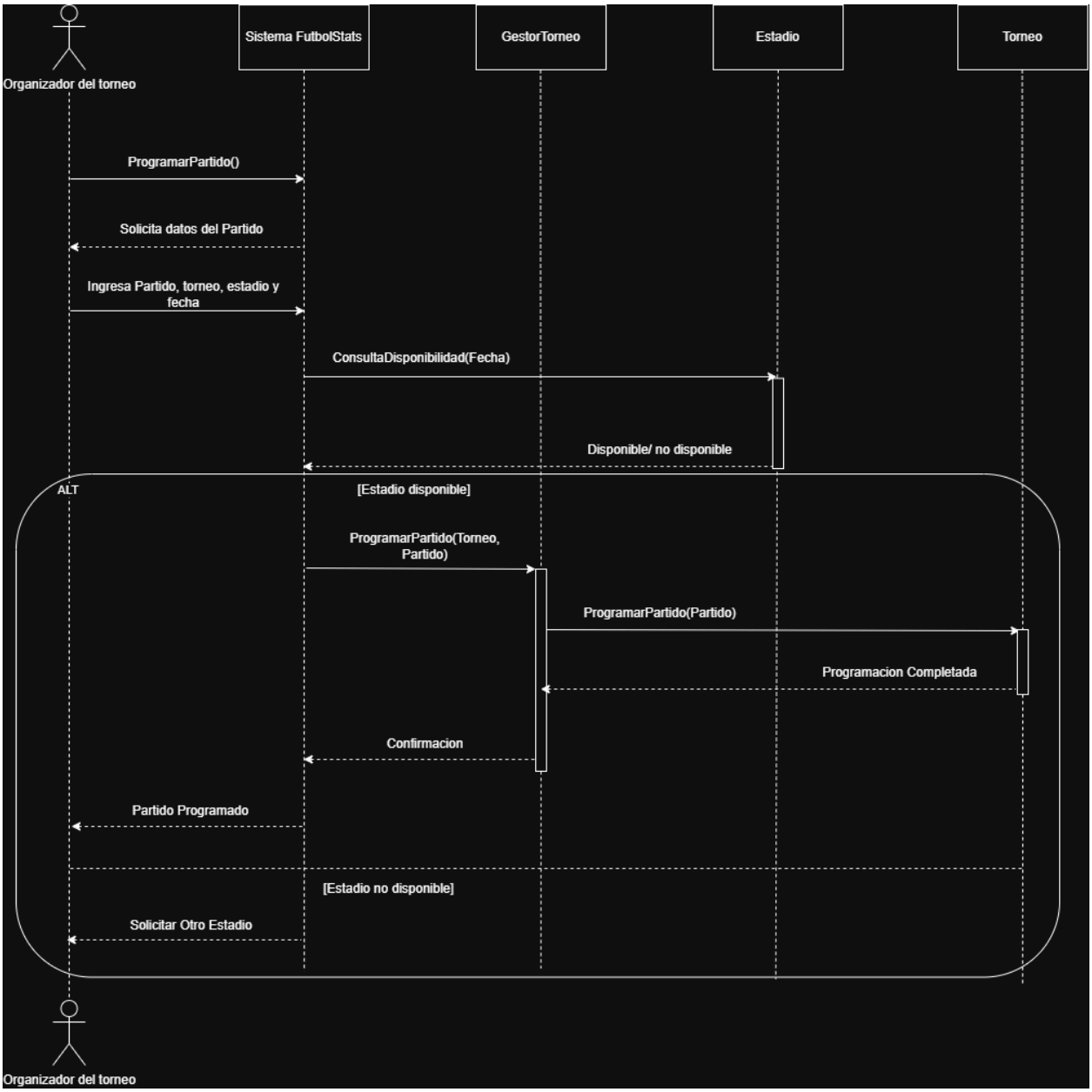
1)



Este diagrama representa las interacciones necesarias para registrar un gol en el sistema FutbolStats.

El proceso comienza cuando el Operador del Partido solicita registrar un gol. El sistema le solicita los datos correspondientes y, una vez ingresados, crea un objeto de la clase `Gol`. Luego, se comunica con la clase `Partido` mediante el método `registrarEvento()`, que permite incorporar el gol al partido. Posteriormente, se obtiene la descripción del evento y se confirma el registro al operador.

2)



Este diagrama representa el proceso de programación de un partido en el sistema FutbolStats.

El proceso comienza cuando el Organizador del Torneo solicita programar un partido e ingresa los datos correspondientes. El sistema consulta la disponibilidad del estadio y, mediante un bloque condicional `alt`, determina cómo continuar.

Si el estadio está disponible, el sistema solicita a `GestorTorneo` que coordine la programación, delegando la operación a la clase `Torneo`. Una vez completada, se confirma al organizador que el partido fue programado. En caso contrario, se solicita seleccionar otro estadio.